

Obligatorisk oppgave nr.2

PROG1001 - Grunnleggende programmering, høsten 2026

Innleveringsfrist: 6.oktober 2026 kl.11:00 (må overholdes)

i Canvas på pdf-format

(Code::Blocks: File → Export → As PDF (uten linjenumre!))

Det skal lages et program som kan lagre opptil 20 brukere (deres navn og *krypterte* passord). Programmet *skal i hvert fall* inneholde følgende `const`'er, to stk. 2-dimensjonale `char`-arrayer og en `int` for å holde orden på antall brukere hittil (du trenger sikkert en god del andre variable også):

```
const int STRLEN      = 40;      ///< Max. tekstlengde.
const int MAXBRUKERE  = 20;      ///< Max. antall brukere.
const int ASCIINRFORST = 33;     ///< Første lovlige ASCII-nr som brukes.
const int ASCIINRSIST = 126;     ///< Siste lovlige ASCII-nr som brukes.
const int MOD = (ASCIINRSIST-ASCIINRFORST)+1; ///< Intervallets lengde.
const int CIPHER      = 42;     ///< Antall høyre-skift ved kryptering.
```

Og inni `main`:

```
char brukerNavn[MAXBRUKERE][STRLEN]; // 2-dim array for brukernavnene.
char brukerPass[MAXBRUKERE][STRLEN]; // 2-dim array for passordene.
int antBrukere = 0;                    // Antall brukere registrert hittil.
```

Krypteringsformen for passordene som vi skal bruke, er den meget enkle «Cæsarchiffer» (mye mer om denne til slutt i dette dokumentet).

Programmet *skal* kunne håndtere følgende fire kommandoer:

(For hver gang programmet skal håndtere en kommando/valg fra brukeren, *skal* det skrives en meny med de lovlige valgene (det kommer en egen feilmelding hver gang noe annet skrives). Store og små kommando-bokstaver skal telle likt (bruk `toupper`).)

N - Ny bruker

Om det allerede er fullt med brukere, kommer det en egen melding. I motsatt fall spør programmet om et brukernavn og passord. Du trenger *ikke* å sjekke for duplikate brukernavn. Det skal kontrolleres at passordet inneholder *minst en* stor bokstav (A-Z), *minst en* liten bokstav (a-z), samt *minst ett* annet *lovlig* tegn (mer i neste avsnitt). Det kommer egne meldinger om passordet *ikke* tilfredsstillende, eller at det inneholder tegn utenfor det lovlige intervallet. Er passordet derimot gyldig, krypteres dette (mer til slutt i dokumentet), og brukerens navn og det krypterte passordet lagres i de to `char`-arrayene. Antall brukere telles så opp.

Passordet kan stort sett inneholde alle de vanlige tegnene du kan skrive inn på tastaturet (unntatt æøå/ÆØÅ), dvs tegnene: `!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~`
Disse er i ASCII-tabellen tegnene fra `'!` (nr.33) til og med `'~` (nr.126). Jfr. `const`'er ovenfor.

L - Logg inn (dvs. sjekke om innskrevet brukernavn og passord stemmer med noe av det lagrede)

Først leses det inn et brukernavn og passord. Passordet blir så kryptert. Deretter sjekkes det om noen allerede lagrede brukere har *både* dette navnet og det krypterte passordet. Finnes en slik match, kommer det en enkel beskjed om at vedkommende er «Logget inn», ellers kommer en melding om at noe er feil med brukernavnet og/eller passordet.

S - Skriv alle brukerne

Er det ikke registrert noen brukere, kommer det en egen melding. I motsatt fall skrives alle brukernes nummer (nummerert fra 1 og oppover), deres navn og krypterte passord - *en* bruker pr.linje.

Q – Avslutt/Quit

Det skrives en melding om at programmet avsluttes, og dette skjer.

«Cæsarchiffer»-kryptering:

Algoritmen for denne er *meget* enkel: hvert enkelt tegn i hele teksten erstattes med et annet tegn som kommer et fast antall (jfr. `const`'en `CHIPHER`) tegn etter vedkommende tegn i ASCII-alfabetet.

F.eks: Om vi bruker bare tegnene 'A'-'Z', teksten er "ARSENAL" og alle tegn erstattes med tegnet som kommer tre plasser lengre ut i alfabetet, så blir den krypterte teksten "DUVHQDO". Ved et slikt tillegg *kan* det skje at vi da havner utenfor det aktuelle tegnintervallet. Da må det «wrappes» om, og starte på begynnelsen igjen av alfabetet. F.eks: Teksten "STUDENT" med et tillegg på syv gir "ZABKLUA".

Algoritmen (litt mer detaljert og konkret):

1. Finn et tegn sin ASCII-verdi.
 2. Trekk fra `ASCIINRFORST` (ASCII-verdien for første lovlig tegn i intervallet).
 3. Pluss på nøkkelen/`CHIPHER` (som er den aktuelle endringen).
 4. Utfør modulus (`MOD`) i tilfelle vi havner utenfor gyldig område, og har behov for å wrappe.
- Hittil er dette koden: `verdi = ((int)tegn - ASCIINRFORST + CHIPHER) % MOD;`
5. Legg til `ASCIINRFORST` igjen, slik at vi får et tegn i det rette intervallet.
- Koden for sluttresultatet: `kryptertTegn = (char)(verdi + ASCIINRFORST);`

To eksempler (om intervallet kun er 'A'-'Z' (ASCII-verdiene 65-90), altså `ASCIINRFORST` = 65, dette er totalt 26 tegn (= `MOD`) og vi lar økningen/`CHIPHER` = 7):

Eks1: 1. og 2.: 'F' (nr.70) - `ASCIINRFORST` = 5

3.: $5 + \text{CHIPHER} = 12$

4.: $12 \% \text{MOD} = 12$

5.: $12 + \text{ASCIINRFORST} = 77$ Altså Cæsar-kryptert til 'M'

Eks2: 1. og 2.: 'X' (nr.88) - `ASCIINRFORST` = 23

3. $23 + \text{CHIPHER} = 30$

4. $30 \% \text{MOD} = 4$

5. $4 + \text{ASCIINRFORST} = 69$ Altså Cæsar-kryptert til 'E'

Tips: Du får nok i hvert fall bruk `bool`, `toupper`, `strcpy`, `strcmp` og `strlen` fra h-filer.

NB: Husk god og *korrekt* (ellers risikerer den å ikke bli godkjent) kommentering, innrykk, leselighet og at linjene ikke blir for lange (slik at teksten wrapper ved en evt. utskrift).

Lykke til!

André Revå Marthinsen, Øystein Qvigstad & FrodeH